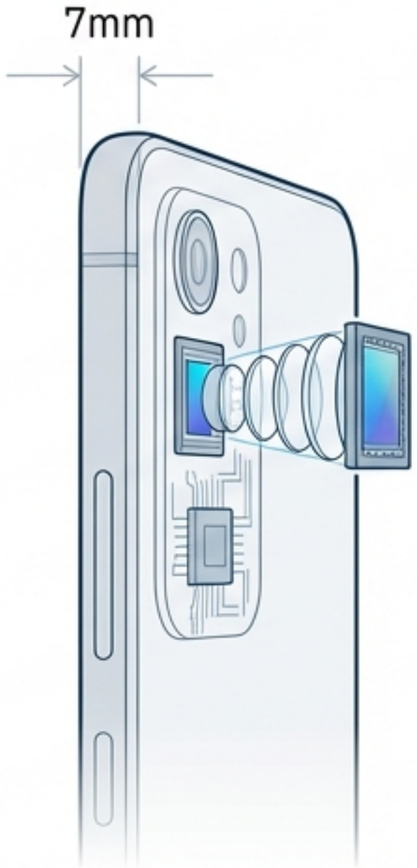
Computational Zoom: Multi-Camera Fusion Implementation

Overcoming the ‘Thin Device’ Paradox with Computer Vision, Deep Learning, and 3D Reconstruction



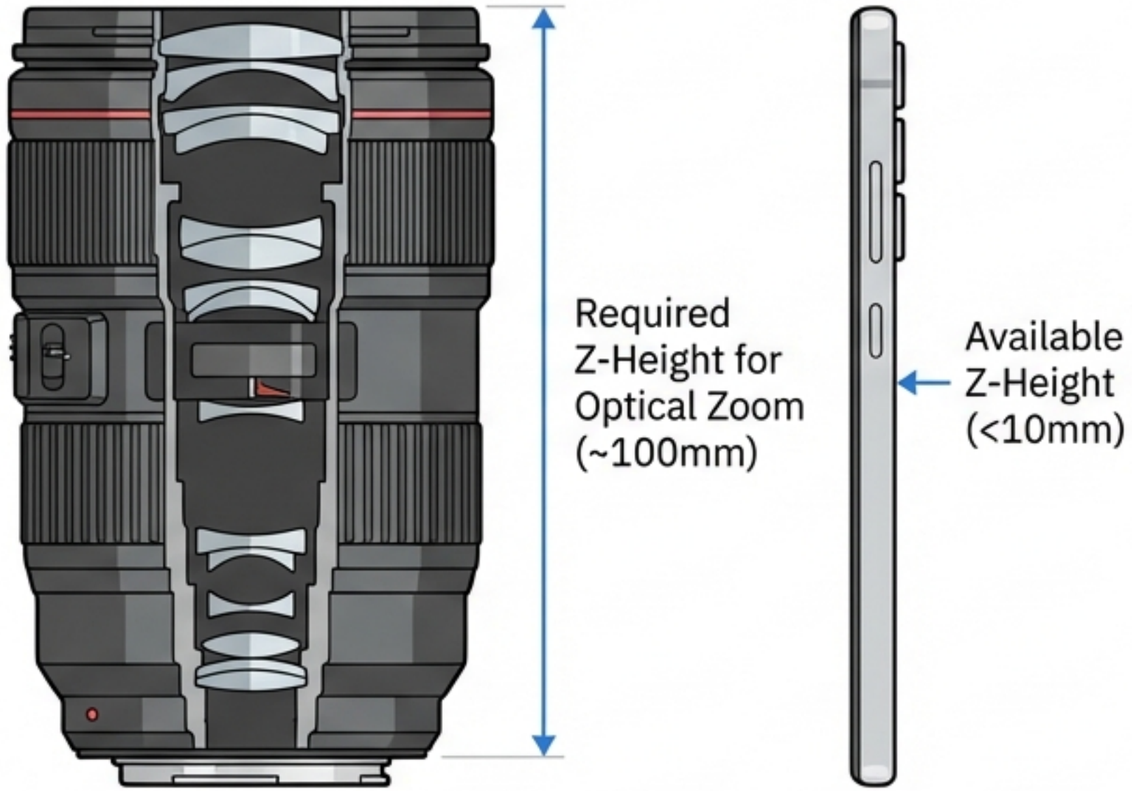


Based on Chapter 15 of Mobile Computational Photography

NotebookLM

The Optical Constraint & The Digital Zoom Failure

(The Constraint)



Physics dictates that long focal lengths require physical depth for moving glass elements.

(The Failure)



Original Photo



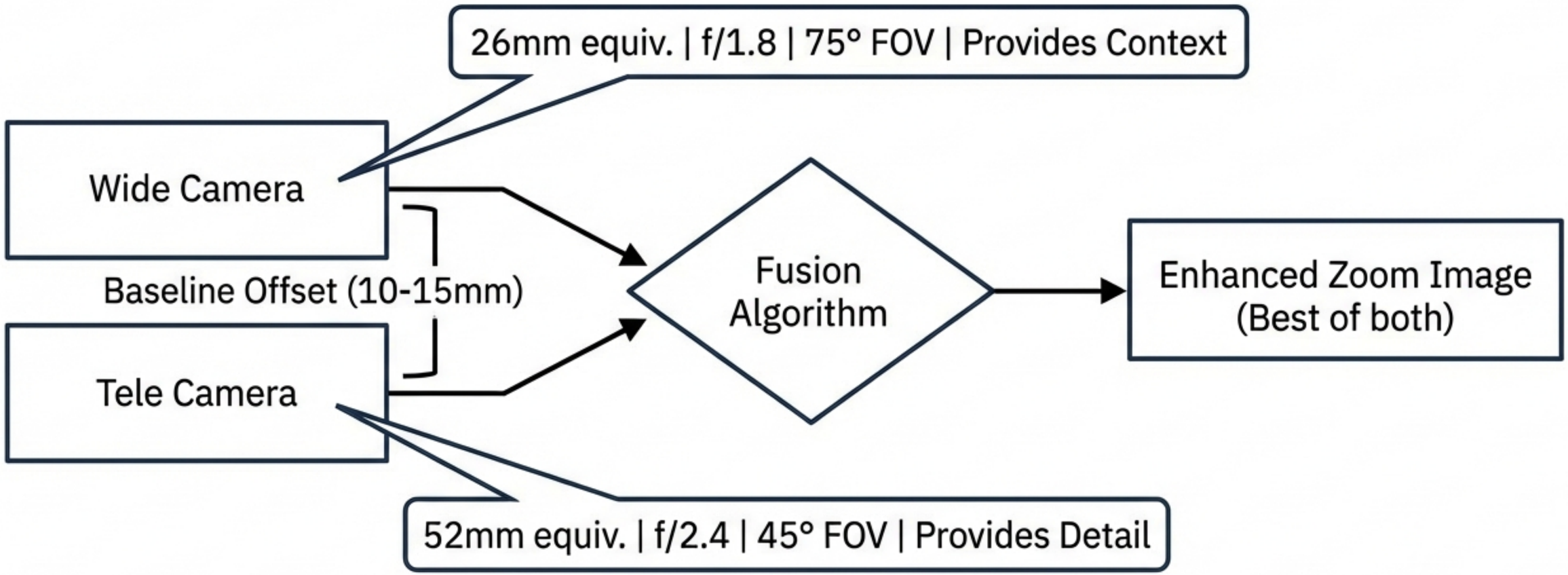
Digital Zoom (Crop + Upscale)

```
cv2.resize(cropped, (w, h), interpolation=cv2.INTER_CUBIC)
```

Result: Severe degradation, "blocky" artifacts, no new detail captured.

NotebookLM

The Dual-Camera Hardware Architecture

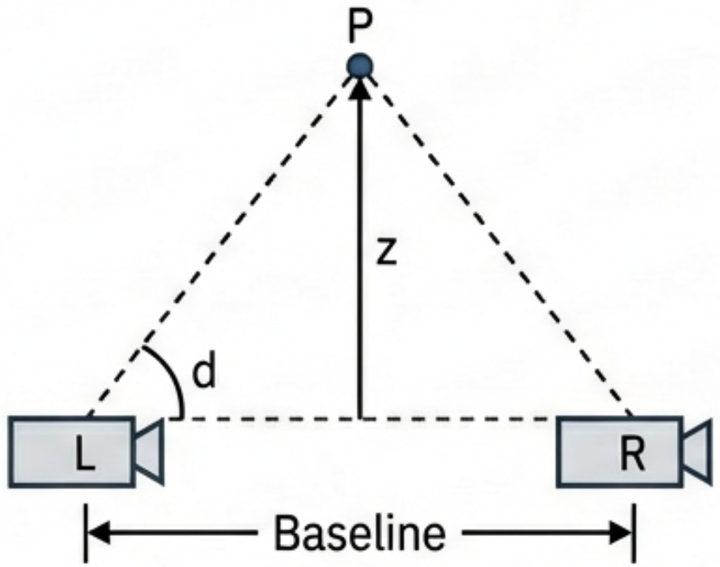


System Logic: The Wide lens provides the color reference and context, while the Telephoto lens captures high-frequency texture details for the center of the image. The baseline offset creates parallax, which is the primary challenge for fusion.

NotebookLM

Core Implementation Challenges

Challenge 1: Geometric Misalignment (Parallax)



$$d = \frac{\text{baseline} \times \text{focal_length}}{z}$$

Objects at different depths shift positions.
Closer objects have larger disparity.
Simple overlaying causes “ghosting”.

Challenge 2: Photometric Differences



Swatch A (Wide)



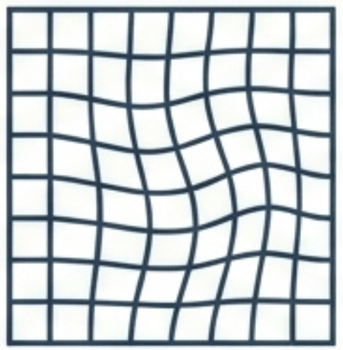
Swatch B (Tele)

- Exposure Disparity
- White Balance / Color Temperature
- Lens Shading / Vignetting
- Sensor Response Curves

NotebookLM

Three Generations of Fusion Algorithms

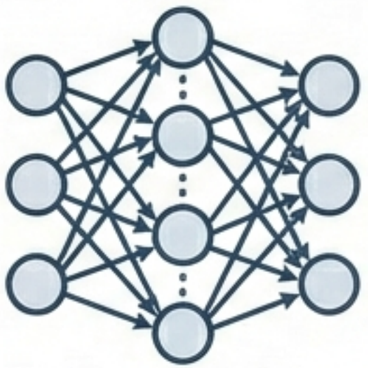
Gen 1: Classical Computer Vision



Feature Matching & Frequency Domain Fusion.

CPU Only | Fast (~1s) | Planar Assumption

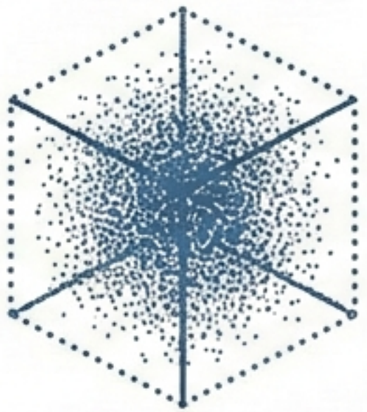
Gen 2: Deep Learning (DCSR)



Aligned Attention Modules.

GPU Required | High Quality Learned Features

Gen 3: 3D Reconstruction (ZoomGS)

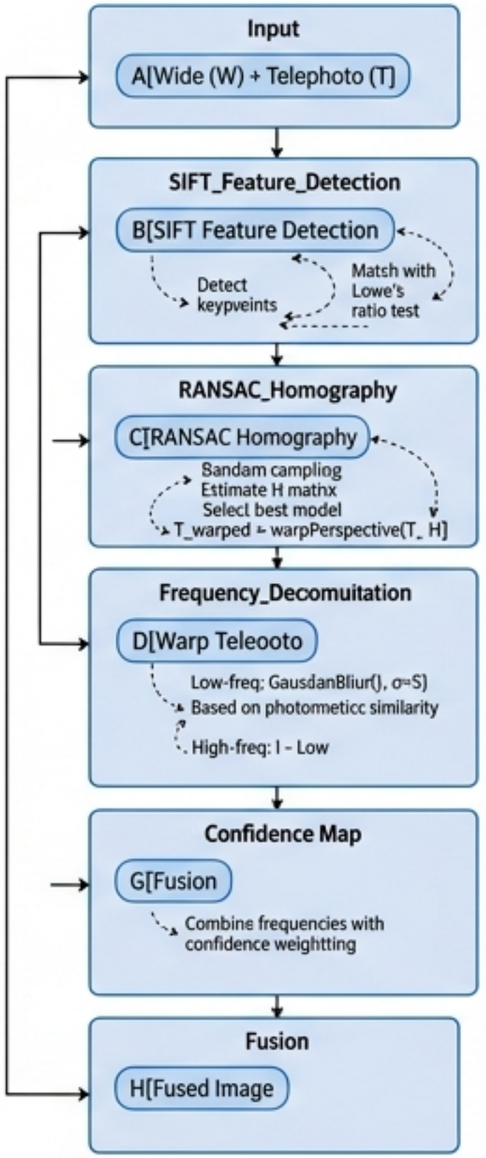


Gaussian Splatting.

Heavy Compute | Continuous Zoom | 3D Volumetric

NotebookLM

Approach I: Classical Frequency Domain Fusion



Why Frequency Domain?

We decompose the images. We take the ‘Color’ (Low Frequency) from the Wide camera and transplant the ‘Sharp Edges’ (High Frequency) from the Telephoto camera. This transplants detail without introducing color artifacts.

NotebookLM

Implementation: Geometric Alignment

```
# 1. Detect Features
sift = cv2.SIFT_create()
kp_wide, des_wide = sift.detectAndCompute(img_wide, None)
kp_tele, des_tele = sift.detectAndCompute(img_tele, None)

# 2. Match with Lowe's Ratio Test
bf = cv2.BFMatcher()
matches = bf.knnMatch(des_wide, des_tele, k=2)
good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

# 3. Compute Homography (RANSAC)
src_pts = np.float32([kp_wide[m.queryIdx].pt for m in good]).reshape(-1,1,2)
dst_pts = np.float32([kp_tele[m.trainIdx].pt for m in good]).reshape(-1,1,2)
H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)

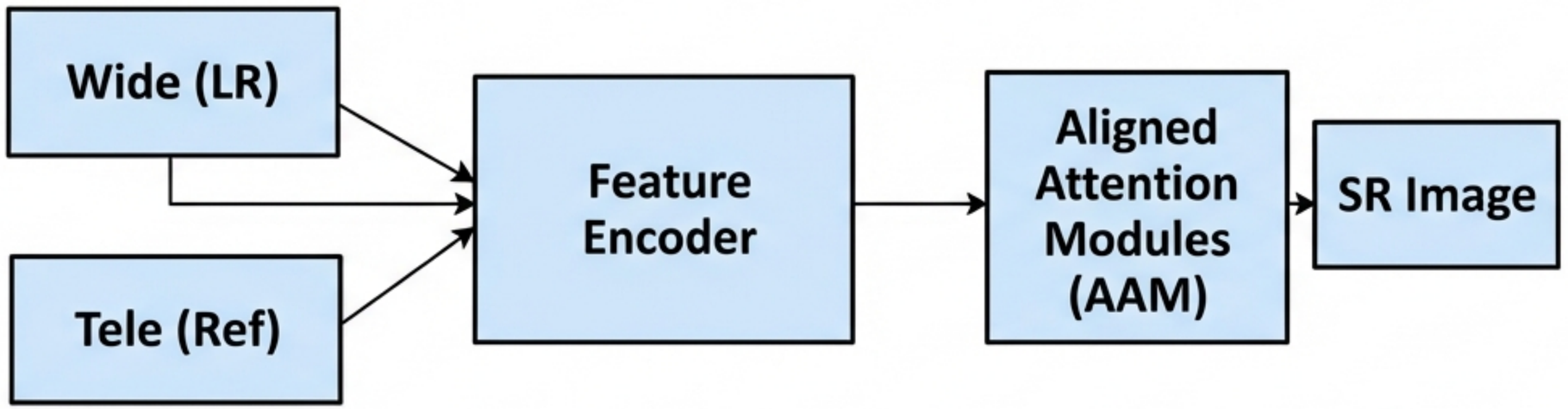
# 4. Warp Telephoto to match Wide
tele_aligned = cv2.warpPerspective(img_tele, H, (w, h))
```

Limitation: Ideally, H is a 3x3 matrix. However, a single matrix assumes the world is a flat plane. Objects popping out of the plane will not align perfectly.

NotebookLM

Approach II: Deep Learning (DCSR)

Dual-Camera Super-Resolution with Aligned Attention



The Problem

Classical homography is a global warp. It fails when scene depth varies locally.

The Solution

DCSR uses an Attention Mechanism. It searches for features pixel-by-pixel, allowing for non-rigid alignment.

The Architecture

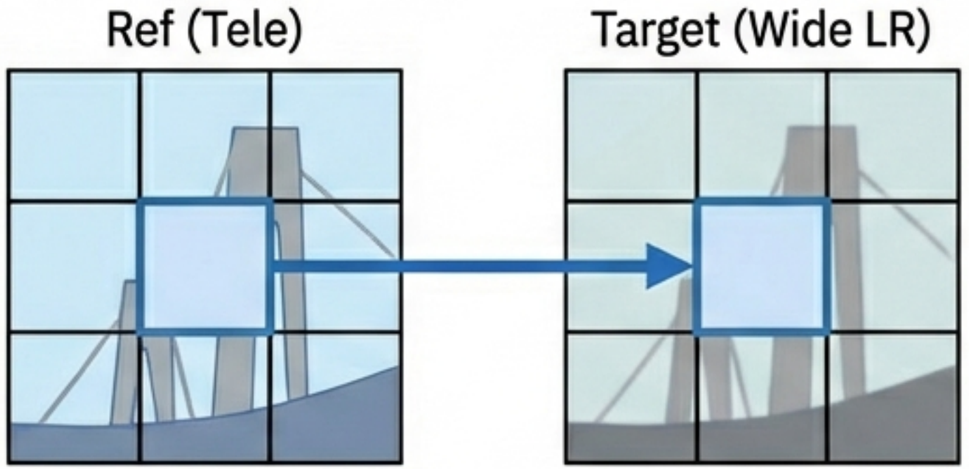
A Siamese ResNet extracts features, which are then fed into the Aligned Attention Module (AAM) to swap information.

NotebookLM

"Inside the Aligned Attention Module (AAM)" in Inter Tight

“Coarse-to-Fine”

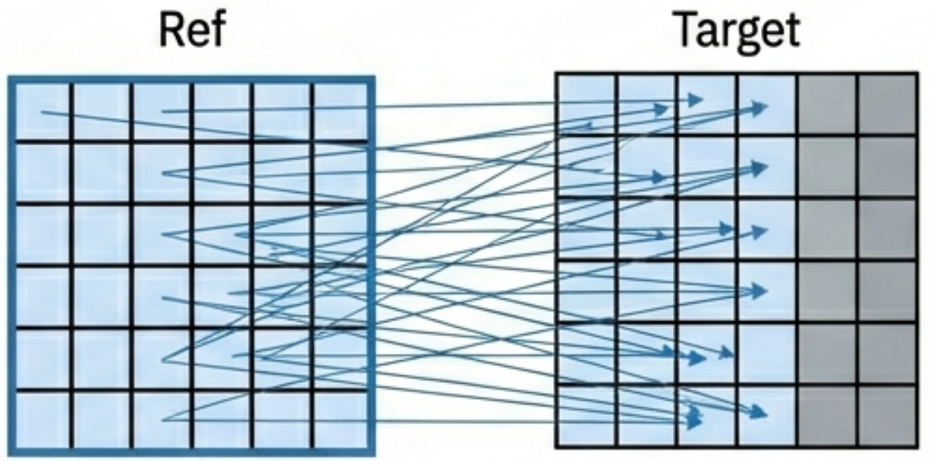
Step 1: Coarse Alignment



Patch Matching
(hard attention)

```
similarity = correlation(F_LR, F_Ref)
```

Step 2: Fine Alignment



Pixel Alignment
(soft attention)

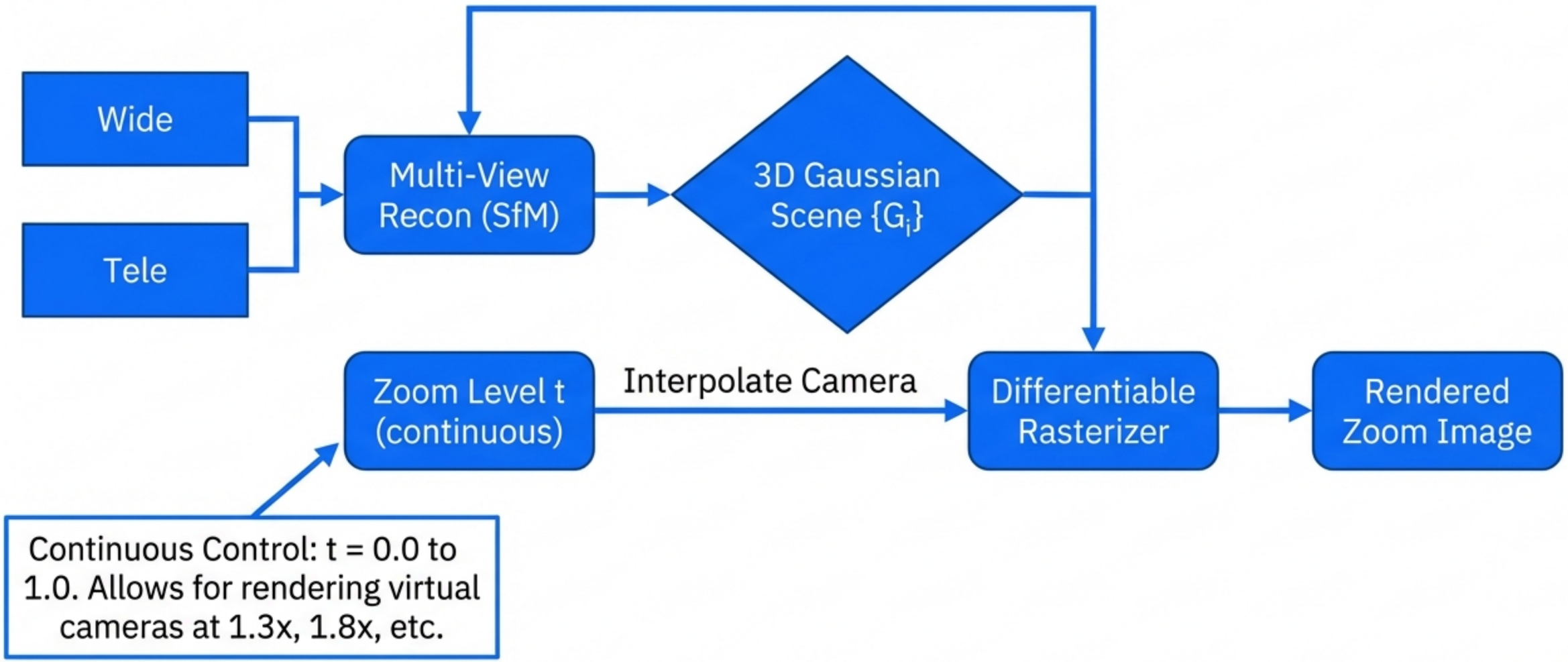
```
F_aligned = sum(AttentionWeights * F_Ref)
```

Result: The network "learns" to fix parallax errors that mathematical models miss, achieving ~32dB PSNR.

NotebookLM

Approach III: 3D Reconstruction (ZoomGS)

Solving the ‘Jump’ with Continuous Zoom

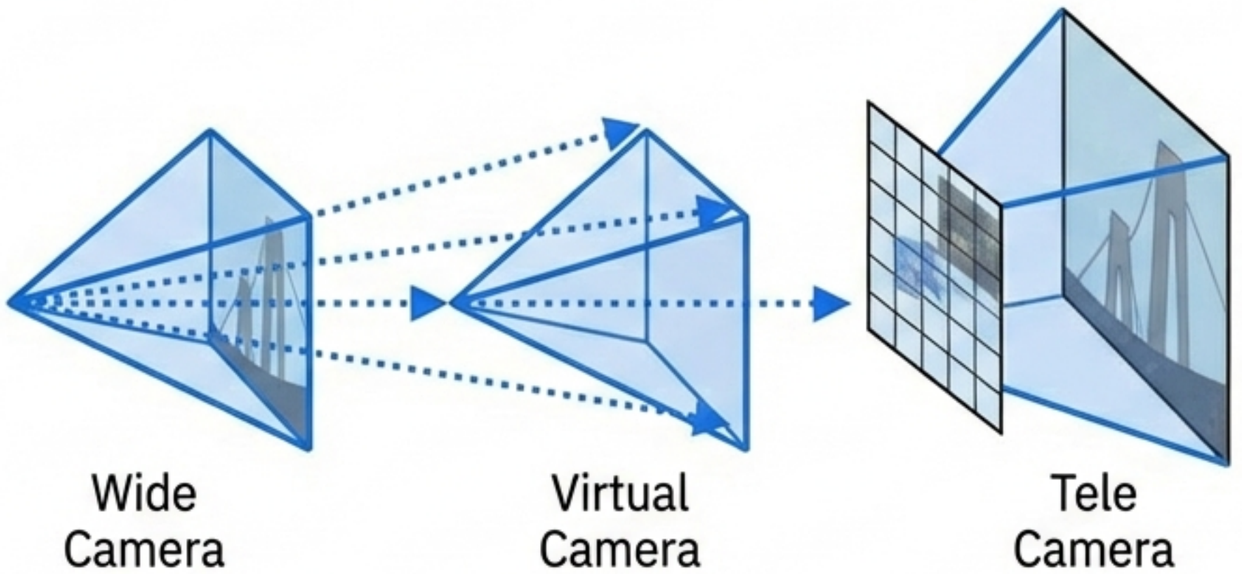


NotebookLM

Rendering Reality: The ZoomGS Pipeline

Gaussian Point Definition

```
class Gaussian3D:
    position: (x, y, z)
    scale:    (sx, sy, sz)
    rotation: quaternion
    opacity:  alpha
    color:    spherical_harmonics
```



By treating the scene as a cloud of 3D particles (Gaussians), we can physically move a virtual camera through the space. The differentiable rasterizer projects these 3D blobs onto a 2D plane to create the new image.

NotebookLM

Comparative Analysis: The Showdown

Feature	Classical (CV)	Deep Learning (DCSR)	ZoomGS (3D)
Core Tech	Feature + Freq	Attention Alignment	Gaussian Splatting
PSNR Quality	28–32 dB	30–35 dB	**32–38 dB**
Speed	**~1 sec (CPU)**	~5 sec (GPU)	~8 sec (GPU)
Memory	**Low (~0.2 GB)**	Medium (~2 GB)	High (~4 GB)
Parallax Handling	Fails on depth	Good	**Excellent**
Zoom Experience	Discrete Steps	Discrete Steps	**Continuous / Smooth**

NotebookLM

Scene-Specific Recommendations



Planar / Document

Use Classical CV. Fast, efficient, and homography is mathematically perfect for flat planes.



Portrait / Depth

Use DCSR. Classical methods will ghost. DCSR handles the parallax around the subject intelligently.



Video Zoom

Use ZoomGS. Essential for smooth transitions. Prevents the jarring ‘pop’ between lenses.



Low Power / IoT

Use Classical CV. The only viable method for standard CPUs without heavy NPU acceleration.

NotebookLM

Getting Started: The Implementation Suite

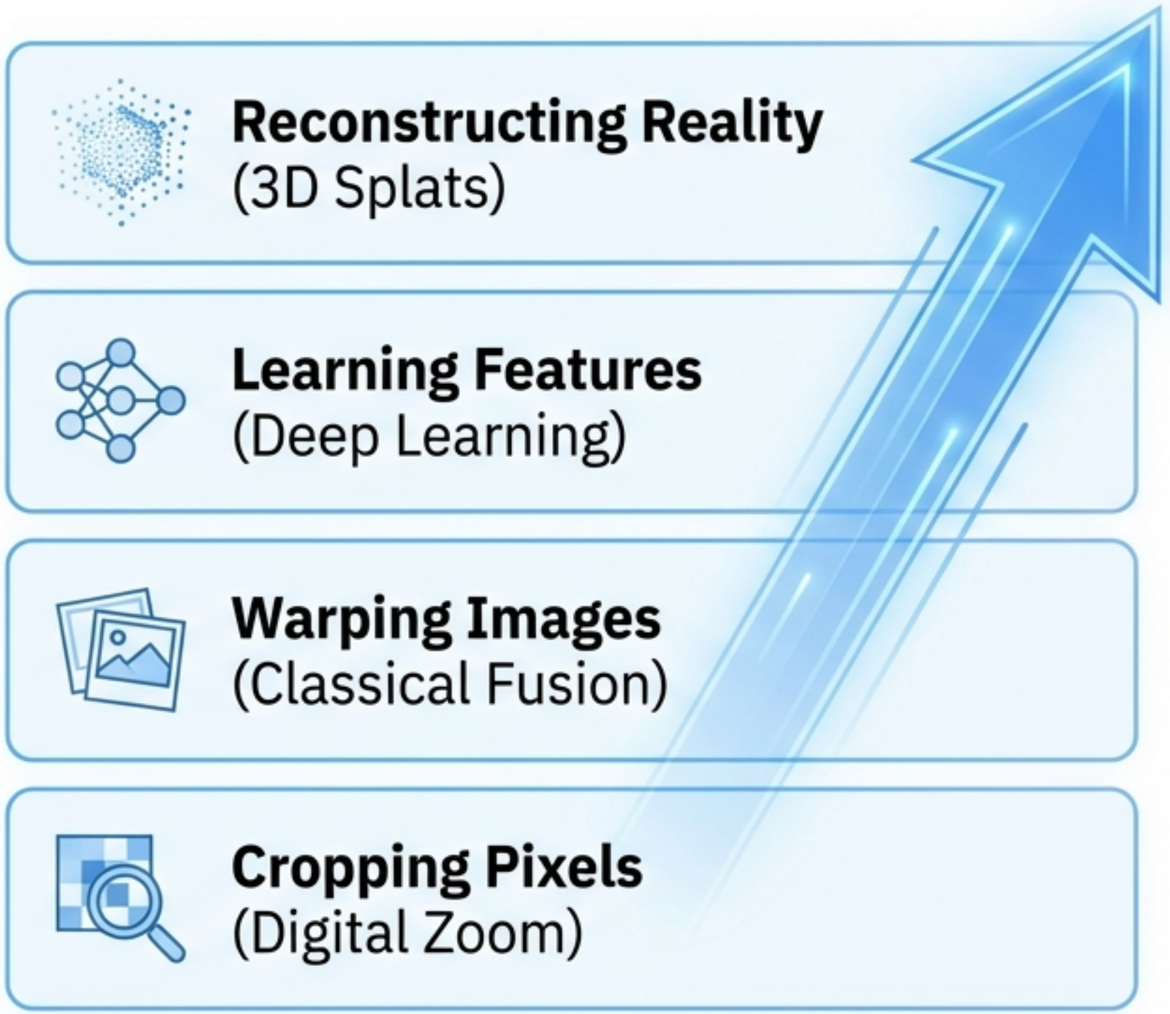
```
$ git clone github.com/JeffRen1977/multicamera
$ cd multicamera
$ python3 -m venv venv
$ source venv/bin/activate
$ pip install -r requirements.txt

# Run the full comparative pipeline
$ ./run_quick_test.sh
> Running Classical Fusion... Done.
> Running DCSR Inference... Done.
> Running ZoomGS Optimization... Done.
> Generating Report: fusion_result.jpg
```



NotebookLM

“The Future of Mobile Zoom” in Inter Tight



Software is the new lens.

The physical limitations of mobile hardware are no longer the ceiling for image quality; they are merely the starting point for computational reconstruction.

NotebookLM